

Course Review: Data Structures and Algorithms, Tied Together

Programming and Data Structures — Week 15, Course Review

Dr. Innocent Nyalala

Objectives

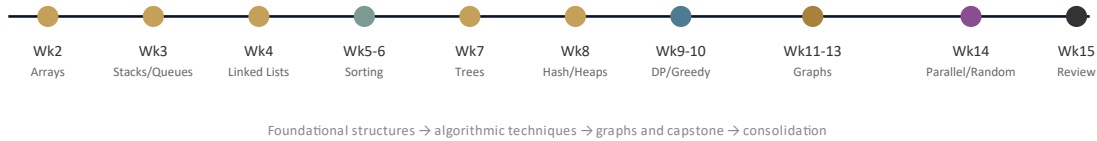
By the end of this review, you will be able to trace the arc of the whole course, from raw memory layout in Week 2 through graphs and randomized algorithms in Weeks 11 through 14; identify which specific data structure or algorithmic technique from this course applies to a new, previously unseen problem; explain how ideas introduced early in the course resurface, transformed, in later weeks; and articulate what your own project presentation will demonstrate, and how it connects to at least two of this course's modules.

Outline

This review proceeds through the course's overall arc presented as a single picture; Part 1 covering data structures from Weeks 2 through 4 and 6 through 8, and how each builds on the one before it; Part 2 covering algorithmic techniques from Week 5 and Weeks 9 through 10 — sorting, dynamic programming, and greedy algorithms; Part 3 covering graphs and the capstone algorithms of Weeks 11 through 14; the recurring themes visible across the entire course; a worked walkthrough of choosing the right tool for a new, unfamiliar problem; project presentation expectations; and closes with exam preparation guidance.

The course's arc, in one picture

Course arc: Weeks 2 through 14



The course's fifteen weeks form a deliberate arc: Weeks 2 through 4 established the foundational data structures — arrays, stacks and queues, and linked lists — that everything after depends on. Weeks 5 and 6 introduced sorting, the first substantial algorithmic technique, progressing from simple $O(n^2)$ methods to $O(n \log n)$ divide-and-conquer methods. Weeks 7 and 8 introduced trees, hash tables, and heaps, which are themselves built from Week 2's arrays and Week 4's linked structures. Weeks 9 and 10 introduced dynamic programming and greedy algorithms, two contrasting strategies for optimization problems. Weeks 11 through 13 introduced graphs, which generalize trees, and the shortest-path and minimum-spanning-tree algorithms built on top of them. Week 14 closed the algorithmic core with parallel and randomized algorithms, showing that even the sequential, deterministic assumptions underlying every prior week can themselves be relaxed.

Part 1: Data structures, building on each other

Week 2 introduced arrays, built from contiguous memory, offering $O(1)$ indexed access and forming the literal foundation that nearly every later data structure in this course is built upon in some form. Week 3 introduced stacks and queues as restricted-access patterns layered on top of either an array or a linked list. Week 4 introduced linked lists, which trade away arrays' $O(1)$ indexed access in exchange for $O(1)$ insertion and deletion at any known position. Week 7 introduced binary trees, which are fundamentally linked structures, but organized into a branching shape specifically to support $O(\log n)$ search, in contrast to Week 4's flat, linear chain. Week 8 introduced hash tables and heaps, both of which return to arrays as the underlying storage, but layer either a computed index via hashing, or an implicit tree shape via array-based heaps, on top of that array.

Part 2: Algorithmic techniques, contrasted

Weeks 5 and 6 covered sorting, progressing from simple $O(n^2)$ methods — bubble, selection, and insertion sort — to divide-and-conquer $O(n \log n)$ methods, merge sort and quicksort. Week 9 introduced dynamic programming, which solves overlapping subproblems exactly once, remembers each answer, and builds up systematically to the full solution. Week 10 introduced greedy algorithms,

which make the locally best choice available at every step and rely on an exchange argument to prove this local choice leads to a globally optimal answer. The recurring contrast between these two techniques is that dynamic programming explores multiple possibilities and remembers the results, while greedy commits to a single choice and never revisits it — and the correct choice between the two techniques depends entirely on whether the problem actually possesses the greedy-choice property, which Week 10’s counterexamples showed cannot simply be assumed.

Part 3: Graphs and the capstone algorithms

Week 11 introduced graph fundamentals: representing graphs via adjacency matrices or adjacency lists, breadth-first and depth-first search, cycle detection, and topological sort — graphs themselves generalize the tree structures introduced in Week 7, since a tree is simply a graph with no cycles. Weeks 12 and 13 introduced shortest-path algorithms — Dijkstra’s, A*, and Bellman-Ford — and minimum spanning tree algorithms, Prim’s and Kruskal’s, the latter being a direct application of Week 10’s greedy algorithms to graph structures rather than a wholly new technique. Week 14 closed the algorithmic core of the course with parallel and randomized algorithms, relaxing the sequential, fully deterministic assumption that every algorithm covered in every prior week had implicitly relied upon.

Recurring theme 1: measure, don’t assume

The theme of measuring rather than assuming recurred throughout the course. Week 2 measured dynamic array resizing’s amortized cost directly, rather than simply asserting it is efficient. Weeks 12 and 13 caught a genuine bug in a hand-traced minimum spanning tree example by re-running the actual code rather than trusting hand computation. Week 14 insisted on measuring real parallel speedup rather than assuming that adding more workers always yields a proportional speedup. This theme applied not only to the course’s algorithmic content but to its own construction: every worked example presented in every lecture throughout this course was independently verified against actual running code before being shipped.

Recurring theme 2: trade-offs, not universally “best” answers

No data structure or algorithm covered in this course is universally “best” — each involves a genuine trade-off. Arrays trade indexed access speed against linked lists’ fast insertion and deletion, from Week 4. Merge sort’s guaranteed $O(n \log n)$ bound trades against quicksort’s usually-faster-in-practice performance with a rare worst-case, from Weeks 5 and 6. Dynamic programming’s guaranteed correctness trades against greedy algorithms’ speed, which is only correct when the specific problem possesses the greedy-choice property, from Weeks 9 and 10. Las Vegas algorithms’ guaranteed correctness trades against Monte Carlo algorithms’ speed with a small, controllable error chance, from Week 14. Recognizing these trade-offs, rather than seeking one algorithm to memorize as universally correct, is the central skill this course has aimed to build.

Choosing the right tool: a worked walkthrough

Consider an unfamiliar problem: given a continuous stream of stock prices, report the median price seen so far after every new price arrives. The first question to ask is whether a single, one-time selection is sufficient — it is not, since the query repeats indefinitely as new data keeps arriving, which rules out a one-shot application of quickselect from Week 14. The second question is whether the full sorted order of every price is needed at every point — it is not, since only the median value is ever actually requested. This points toward the two-heap technique from Week 8: one heap holding the smaller half of the data seen so far and one holding the larger half, rebalanced after every insertion so their sizes never differ by more than one, with the median always available at one of the two heaps' roots in $O(\log n)$ time per insertion. This walkthrough illustrates the general process of choosing the right tool: identify the actual query pattern first, then match it to the data structure or algorithm whose guarantees fit that pattern.

A second worked walkthrough: “spell-checker suggestions”

Consider a second unfamiliar problem: given a misspelled word, suggest the five closest dictionary words. This requires both a way to measure “closeness” between two strings and an efficient way to search across many candidate words. Dynamic programming's edit distance algorithm, covered in Week 9, computes exactly this closeness measure directly and can be reused without modification. A hash table, from Week 8, or a trie-like prefix structure can narrow the set of candidate dictionary words before computing edit distance against each one individually, avoiding the cost of comparing the misspelled word against every single word in the entire dictionary. This walkthrough illustrates that solving an unfamiliar problem often means composing two or more techniques from different weeks together, rather than finding one single technique that solves the whole problem alone.

Project rubric, at a glance

The project presentation rubric assesses five components. Problem framing checks whether the problem being solved is clearly stated and genuinely non-trivial, not an artificially simple restatement of a lecture example. Technique justification checks whether the chosen data structure or algorithm is explained and compared against a plausible alternative, not merely asserted. Demonstration checks whether the application actually runs successfully on real or realistic input, not only on a single hand-picked toy example. Performance analysis checks that any claimed speed or efficiency is measured directly, not assumed, in keeping with this course's recurring theme. Finally, the question-and-answer component checks whether the presenter can defend the specific trade-offs their design made when questioned directly.

Common project pitfalls, from past cohorts

Several pitfalls have recurred across past project cohorts. The most common is choosing a technique simply because it was covered in the most recent lecture, rather than because it genuinely fits the problem at hand — the tool-selection reasoning process demonstrated earlier in this lecture is the corrective for this pitfall. Another common pitfall is claiming a performance improvement without ever measuring the baseline being compared against, which makes the claimed improvement unverifiable. A third is presenting a working demonstration without any discussion of what would break, or slow down disproportionately, at ten or a hundred times the scale actually shown in the demonstration.

Exam format and structure

The exam will include a mix of question types: short-answer complexity analysis questions, a hand-trace of a named algorithm from the course, and at least one scenario question requiring the “choose the right tool” reasoning process demonstrated in this lecture’s two worked walkthroughs. Preparation should focus on the reasoning process itself, not merely memorizing the final answer to each walkthrough. Complexity analysis questions specifically test understanding of why a bound holds — for instance, quickselect’s geometric-series argument from Week 14 — rather than only the memorized asymptotic bound in isolation.

Sample exam-style question

As a sample exam-style question: a ride-sharing application needs to find the fastest route between a driver and three nearby ride requests, where road travel times between locations are known and non-negative, and a reasonable straight-line-distance estimate to each request location is available. Identify which algorithm from this course applies, and explain precisely why, referencing the specific conditions (non-negative weights, an available admissible heuristic) that make that algorithm the correct choice over its alternatives.

Sample question — answer

The correct answer is A* search, from Week 12. The non-negative edge weights rule out needing Bellman-Ford’s negative-weight handling from Week 13, and critically, a usable straight-line-distance heuristic is explicitly available in the scenario. A* uses that heuristic to explore fewer nodes than plain Dijkstra’s search while still guaranteeing the shortest path is found, provided the heuristic is admissible, meaning it never overestimates the true remaining distance. Plain Dijkstra’s algorithm would also produce a correct answer on this scenario, but A* is the better choice specifically because the scenario supplies an available heuristic — recognizing when a question’s specific details should change which algorithm is preferred, not merely which algorithm would technically work, is exactly the skill being tested.

Project presentation expectations

Project presentations are expected to demonstrate a real application combining at least two of this course's modules — for instance, a route-planning application combining Weeks 11 through 13's graph algorithms with Week 8's priority queue implementation. Each presentation should cover the problem being solved, the specific data structure or algorithm choices made and the reasoning behind them, a live or recorded demonstration of the working application, and a measured performance analysis rather than an assumed one, in keeping with this course's recurring "measure, don't assume" theme. Presenters should be prepared to answer what trade-off their design made, and what they would do differently given more time.

A note on partial credit and reasoning

On both the exam and the project presentation, demonstrating the reasoning process — why a particular technique was chosen, and what alternatives were considered and ruled out, and why — earns credit even when the final implementation contains bugs or the exam answer contains a minor computational error. This mirrors the way this course itself was constructed: every worked example throughout the fifteen weeks was verified directly against real running code, but the actual educational value was always in the reasoning process that led to each answer, not merely in the final numeric result. Consequently, a technically correct answer presented with no visible reasoning is worth less credit than a well-reasoned answer that contains a minor implementation slip.

Exam preparation guidance

Exam preparation should begin by reviewing each week's Quiz and Exercise slides, since these were specifically written to test the particular distinction each individual lecture introduced. For every algorithm covered in the course, be prepared to state its time complexity, the trade-off it makes against at least one alternative covered elsewhere in the course, and to work through one example by hand. Beyond memorization, practice applying the "choosing the right tool" reasoning process demonstrated in this lecture to two or three genuinely unfamiliar problems, not only ones already seen in lecture or lab material.

Closing note

Fifteen weeks ago, this course opened with a comparison between object-oriented design and algorithmic thinking in Week 1. It closes here having covered every major data structure and algorithmic paradigm in standard, widespread use in professional software engineering today, from arrays through graphs, dynamic programming, and randomized algorithms. The single most transferable skill this course has aimed to build is not any one specific algorithm memorized in isolation, but the discipline of

recognizing a new problem's actual shape and matching it to the right tool, with every claim about that tool's correctness and performance honestly verified rather than assumed.

Summary

This course's arc ran from raw memory in Week 2, through restricted access patterns in Week 3, flexible linking in Week 4, branching structures in Week 7, and computed or implicit structures in Week 8; through sorting in Weeks 5 and 6 and optimization techniques in Weeks 9 and 10; through graphs in Weeks 11 through 13; and finally to relaxing the sequential, deterministic assumption itself in Week 14. Two themes recurred throughout: measuring rather than assuming, and recognizing that every technique covered is a genuine trade-off rather than a universally correct choice. Project presentations should combine at least two of the course's modules and include a genuinely measured, not assumed, performance analysis. Exam preparation should revisit each week's Quiz and Exercise pair and practice the tool-selection reasoning process demonstrated in this lecture on genuinely unfamiliar problems.